

Webmozart Assert

stable 2.0.0 downloads 861.81 M

This library contains efficient assertions to test the input and output of your methods. With these assertions, you can greatly reduce the amount of coding needed to write a safe implementation.

All assertions in the `[Assert]` class throw an `Webmozart\Assert\InvalidArgumentException` if they fail.

FAQ

What's the difference to beberlei/assert?

This library is heavily inspired by Benjamin Eberlei's wonderful assert package, but fixes a usability issue with error messages that can't be fixed there without breaking backwards compatibility.

This package features usable error messages by default. However, you can also easily write custom error messages:

```
Assert::string($path, 'The path is expected to be a string. Got: %s');
```

In beberlei/assert, the ordering of the `%s` placeholders is different for every assertion. This package, on the contrary, provides consistent placeholder ordering for all assertions:

- `%s`: The tested value as string, e.g. `"/foo/bar"`.
- `%2$s`, `%3$s`, ...: Additional assertion-specific values, e.g. the minimum/maximum length, allowed values, etc.

Check the source code of the assertions to find out details about the additional available placeholders.

Installation

Use Composer to install the package:

```
composer require webmozart/assert
```

Example

```
use Webmozart\Assert\Assert;

class Employee
{
    public function __construct($id)
    {
        Assert::integer($id, 'The employee ID must be an integer. Got: %s');
```

```

        Assert::greaterThan($id, 0, 'The employee ID must be a positive integer. Got: %s');
    }
}

```

If you create an employee with an invalid ID, an exception is thrown:

```

new Employee('foobar');
// => Webmozart\Assert\InvalidArgumentException:
//     The employee ID must be an integer. Got: string

new Employee(-10);
// => Webmozart\Assert\InvalidArgumentException:
//     The employee ID must be a positive integer. Got: -10

```

Assertions

The [Assert] class provides the following assertions:

Type Assertions

Method	Description
<code>string(\$value, \$message = '')</code>	Check that a value is a string
<code>stringNotEmpty(\$value, \$message = '')</code>	Check that a value is a non-empty string
<code>integer(\$value, \$message = '')</code>	Check that a value is an integer
<code>integerish(\$value, \$message = '')</code>	Check that a value casts to an integer
<code>positiveInteger(\$value, \$message = '')</code>	Check that a value is a positive (non-zero) integer
<code>float(\$value, \$message = '')</code>	Check that a value is a float
<code>numeric(\$value, \$message = '')</code>	Check that a value is numeric
<code>natural(\$value, \$message = '')</code>	Check that a value is a non-negative integer
<code>boolean(\$value, \$message = '')</code>	Check that a value is a boolean
<code>scalar(\$value, \$message = '')</code>	Check that a value is a scalar
<code>object(\$value, \$message = '')</code>	Check that a value is an object
<code>resource(\$value, \$type = null, \$message = '')</code>	Check that a value is a resource
<code>isCallable(\$value, \$message = '')</code>	Check that a value is a callable
<code>isArray(\$value, \$message = '')</code>	Check that a value is an array
<code>isTraversable(\$value, \$message = '') (deprecated)</code>	Check that a value is an array or a <code>\Traversable</code>
<code>isIterable(\$value, \$message = '')</code>	Check that a value is an array or a <code>\Traversable</code>
<code>isCountable(\$value, \$message = '')</code>	Check that a value is an array or a <code>\Countable</code>

Method	Description
<code>assertInstanceOf(\$value, \$class, \$message = '')</code>	Check that a value is an instanceof a class
<code>assertInstanceOfAny(\$value, array \$classes, \$message = '')</code>	Check that a value is an instanceof at least one class on the array of classes
<code>notInstanceOf(\$value, \$class, \$message = '')</code>	Check that a value is not an instanceof a class
<code>isAOf(\$value, \$class, \$message = '')</code>	Check that a value is of the class or has one of its parents
<code>isAnyOf(\$value, array \$classes, \$message = '')</code>	Check that a value is of at least one of the classes or has one of its parents
<code>isNotA(\$value, \$class, \$message = '')</code>	Check that a value is not of the class or has not one of its parents
<code>isArrayAccessible(\$value, \$message = '')</code>	Check that a value can be accessed as an array
<code>uniqueValues(\$values, \$message = '')</code>	Check that the given array contains unique values

Comparison Assertions

Method	Description
<code>true(\$value, \$message = '')</code>	Check that a value is true
<code>false(\$value, \$message = '')</code>	Check that a value is false
<code>notFalse(\$value, \$message = '')</code>	Check that a value is not false
<code>null(\$value, \$message = '')</code>	Check that a value is null
<code>notNull(\$value, \$message = '')</code>	Check that a value is not null
<code>isEmpty(\$value, \$message = '')</code>	Check that a value is empty()
<code>notEmpty(\$value, \$message = '')</code>	Check that a value is not empty()
<code>eq(\$value, \$value2, \$message = '')</code>	Check that a value equals another (==)
<code>notEq(\$value, \$value2, \$message = '')</code>	Check that a value does not equal another (!=)
<code>same(\$value, \$value2, \$message = '')</code>	Check that a value is identical to another (===)

Method	Description
<code>notSame(\$value, \$value2, \$message = '')</code>	Check that a value is not identical to another (<code>!==</code>)
<code>greaterThan(\$value, \$value2, \$message = '')</code>	Check that a value is greater than another
<code>greaterThanEq(\$value, \$value2, \$message = '')</code>	Check that a value is greater than or equal to another
<code>lessThan(\$value, \$value2, \$message = '')</code>	Check that a value is less than another
<code>lessThanEq(\$value, \$value2, \$message = '')</code>	Check that a value is less than or equal to another
<code>range(\$value, \$min, \$max, \$message = '')</code>	Check that a value is within a range
<code>inArray(\$value, array \$values, \$message = '')</code>	Check that a value is one of a list of values
<code>oneOf(\$value, array \$values, \$message = '')</code>	Check that a value is one of a list of values (alias of <code>inArray</code>)

String Assertions

You should check that a value is a string with `Assert::string()` before making any of the following assertions.

Method	Description
<code>contains(\$value, \$subString, \$message = '')</code>	Check that a string contains a substring
<code>notContains(\$value, \$subString, \$message = '')</code>	Check that a string does not contain a substring
<code>startsWith(\$value, \$prefix, \$message = '')</code>	Check that a string has a prefix
<code>notStartsWith(\$value, \$prefix, \$message = '')</code>	Check that a string does not have a prefix
<code>startsWithLetter(\$value, \$message = '')</code>	Check that a string starts with a letter
<code>endsWith(\$value, \$suffix, \$message = '')</code>	Check that a string has a suffix
<code>notEndsWith(\$value, \$suffix, \$message = '')</code>	Check that a string does not have a suffix
<code>regex(\$value, \$pattern, \$message = '')</code>	Check that a string matches a regular expression
<code>notRegex(\$value, \$pattern, \$message = '')</code>	Check that a string does not match a regular expression
<code>unicodeLetters(\$value, \$message = '')</code>	Check that a string contains Unicode letters only

Method	Description
<code>alpha(\$value, \$message = '')</code>	Check that a string contains letters only
<code>digits(\$value, \$message = '')</code>	Check that a string contains digits only
<code>alnum(\$value, \$message = '')</code>	Check that a string contains letters and digits only
<code>lower(\$value, \$message = '')</code>	Check that a string contains lowercase characters only
<code>upper(\$value, \$message = '')</code>	Check that a string contains uppercase characters only
<code>length(\$value, \$length, \$message = '')</code>	Check that a string has a certain number of characters
<code>minLength(\$value, \$min, \$message = '')</code>	Check that a string has at least a certain number of characters
<code>maxLength(\$value, \$max, \$message = '')</code>	Check that a string has at most a certain number of characters
<code>lengthBetween(\$value, \$min, \$max, \$message = '')</code>	Check that a string has a length in the given range
<code>uuid(\$value, \$message = '')</code>	Check that a string is a valid UUID
<code>ip(\$value, \$message = '')</code>	Check that a string is a valid IP (either IPv4 or IPv6)
<code>ipv4(\$value, \$message = '')</code>	Check that a string is a valid IPv4
<code>ipv6(\$value, \$message = '')</code>	Check that a string is a valid IPv6
<code>email(\$value, \$message = '')</code>	Check that a string is a valid e-mail address
<code>notWhitespaceOnly(\$value, \$message = '')</code>	Check that a string contains at least one non-whitespace character

File Assertions

Method	Description
<code>fileExists(\$value, \$message = '')</code>	Check that a value is an existing path
<code>file(\$value, \$message = '')</code>	Check that a value is an existing file
<code>directory(\$value, \$message = '')</code>	Check that a value is an existing directory
<code>readable(\$value, \$message = '')</code>	Check that a value is a readable path
<code>writable(\$value, \$message = '')</code>	Check that a value is a writable path

Object Assertions

Method	Description
<code>classExists(\$value, \$message = '')</code>	Check that a value is an existing class name
<code>subclassOf(\$value, \$class, \$message = '')</code>	Check that a class is a subclass of another
<code>interfaceExists(\$value, \$message = '')</code>	Check that a value is an existing interface name
<code>implementsInterface(\$value, \$class, \$message = '')</code>	Check that a class implements an interface
<code>propertyExists(\$value, \$property, \$message = '')</code>	Check that a property exists in a class/object
<code>propertyNotExists(\$value, \$property, \$message = '')</code>	Check that a property does not exist in a class/object
<code>methodExists(\$value, \$method, \$message = '')</code>	Check that a method exists in a class/object
<code>methodNotExists(\$value, \$method, \$message = '')</code>	Check that a method does not exist in a class/object

Array Assertions

Method	Description
<code>keyExists(\$array, \$key, \$message = '')</code>	Check that a key exists in an array
<code>keyNotExists(\$array, \$key, \$message = '')</code>	Check that a key does not exist in an array
<code>validArrayKey(\$key, \$message = '')</code>	Check that a value is a valid array key (int or string)
<code>count(\$array, \$number, \$message = '')</code>	Check that an array contains a specific number of elements
<code>minCount(\$array, \$min, \$message = '')</code>	Check that an array contains at least a certain number of elements
<code>maxCount(\$array, \$max, \$message = '')</code>	Check that an array contains at most a certain number of elements
<code>countBetween(\$array, \$min, \$max, \$message = '')</code>	Check that an array has a count in the given range
<code>isList(\$array, \$message = '')</code>	Check that an array is a non-associative list
<code>isNonEmptyList(\$array, \$message = '')</code>	Check that an array is a non-associative list, and not empty
<code>isMap(\$array, \$message = '')</code>	Check that an array is associative and has strings as keys

Method	Description
<code>isEmptyMap(\$array, \$message = '')</code>	Check that an array is associative and has strings as keys, and is not empty

Function Assertions

Method	Description
<code>throws(\$closure, \$class, \$message = '')</code>	Check that a function throws a certain exception. Subclasses of the exception class will be accepted.

Collection Assertions

All of the above assertions can be prefixed with `all*()` to test the contents of an array or a `\Traversable`:

```
Assert::allIsInstanceOf($employees, 'Acme\Employee');
```

Nullable Assertions

All of the above assertions can be prefixed with `nullOr*` to run the assertion only if it the value is not `null`:

```
Assert::nullOrString($middleName, 'The middle name must be a string or null. Got: %s');
```

Extending Assert

The `Assert` class comes with a few methods, which can be overridden to change the class behaviour. You can also extend it to add your own assertions.

Overriding methods Overriding the following methods in your assertion class allows you to change the behaviour of the assertions:

- `public static function __callStatic($name, $arguments)`
 - This method is used to ‘create’ the `nullOr` and `all` versions of the assertions.
- `protected static function valueToString($value)`
 - This method is used for error messages, to convert the value to a string value for displaying. You could use this for representing a value object with a `__toString` method for example.
- `protected static function typeToString($value)`
 - This method is used for error messages, to convert the a value to a string representing its type.
- `protected static function strlen($value)`

- This method is used to calculate string length for relevant methods, using the `mb_strlen` if available and useful.
- `protected static function reportInvalidArgument($message)`
 - This method is called when an assertion fails, with the specified error message. Here you can throw your own exception, or log something.

Static analysis support

Where applicable, assertion functions are annotated to support Psalm's Assertion syntax. A dedicated PHPStan Plugin is required for proper type support.

Authors

- Bernhard Schussek a.k.a. [[@webmozart](#)]
- The Community Contributors

Contribute

Contributions to the package are always welcome!

- Report any bugs or issues you find on the issue tracker.
- You can grab the source code at the package's Git repository.

License

All contents of this package are licensed under the [MIT license].

[[@webmozart](#)]: <https://twitter.com/webmozart> [MIT license]: LICENSE
 [Assert]: src/Assert.php